

Ampliando: Permisos para creadores de contenido

Casos de uso reales y mejores prácticas

JA Palazón Ferrando

2026-06-22

Contenidos

1	Casos de uso reales	2
1.1	Institución educativa	2
1.2	Equipo de publicación	2
1.3	Departamento legal	3
1.4	Uso personal	4
2	Matriz de permisos por rol	4
3	Depuración de permisos	5
3.1	Síntoma: “No tengo permiso para editar”	5
3.2	Síntoma: OpenCode no puede leer ciertos archivos	5
3.3	Síntoma: Bash no funciona aunque está permitido	5
3.4	Tabla de diagnóstico rápido	5
4	Permisos y Git	5
4.1	Flujo de trabajo recomendado	6
4.2	Ejemplo práctico	6
5	Lista de verificación	6
6	Errores comunes	6
7	Referencias	7

Has visto lo básico en el paso 7

Este documento amplía lo que viste en el paso 7. Si no lo has visto aún, te recomiendo empezar por allí.

¿Para quién es esta guía?

Para creadores de contenido que trabajan en equipo o manejan documentos sensibles: profesorado, editores, departamentos legales, o cualquier persona que quiera proteger sus archivos. **No necesitas saber programar** para entenderla.

1. Casos de uso reales

1.1. Institución educativa

Imagina una universidad donde profesores y comparten documentos con estudiantes. Cada rol tiene necesidades diferentes:

Estudiantes — pueden leer materiales y entregar trabajos, pero no modificar los documentos del profesor:

```
{
  "permission": {
    "edit": "deny",
    "write": "allow",
    "bash": "deny",
    "read": {
      "allow": ["materiales/", "entregas/"],
      "deny": ["calificaciones/", "privado/"]
    }
  }
}
```

Profesores — pueden editar sus materiales y revisar entregas, pero no ejecutar comandos del sistema:

```
{
  "permission": {
    "edit": "allow",
    "write": "allow",
    "bash": "deny",
    "read": "allow"
  }
}
```

Administradores — acceso completo para configurar el sistema:

```
{
  "permission": {
    "edit": "allow",
    "write": "allow",
    "bash": "allow",
    "read": "allow"
  }
}
```

Consejo para instituciones

Guarda cada configuración en un archivo `opencode.jsonc` separado por roles. Así puedes reutilizarlos en diferentes proyectos o asignarlos a agentes específicos.

1.2. Equipo de publicación

En una editorial, el flujo de trabajo sigue un camino: Escritor → Editor → Revisor → Director. Cada uno necesita permisos diferentes:

Escritor — crea y modifica borradores:

```
{
  "permission": {
    "edit": "allow",
    "write": "allow",
    "bash": "deny",
    "read": {
      "allow": ["borradores/", "plantillas/"],
      "deny": ["publicados/", "legales/"]
    }
  }
}
```

Editor — revisa y aprueba cambios:

```
{
  "permission": {
    "edit": "allow",
    "write": "allow",
    "bash": "deny",
    "read": "allow"
  }
}
```

Revisor — solo lee y comenta, sin modificar:

```
{
  "permission": {
    "edit": "deny",
    "write": "deny",
    "bash": "deny",
    "read": {
      "allow": ["borradores/", "revisiones/"],
      "deny": ["privado/"]
    }
  }
}
```

Director — acceso completo para decisiones finales:

```
{
  "permission": {
    "edit": "allow",
    "write": "allow",
    "bash": "allow",
    "read": "allow"
  }
}
```

1.3. Departamento legal

Los documentos legales requieren máxima protección. OpenCode puede consultar contratos y normativas, pero nunca debe modificarlos:

```
{
  "permission": {
    "edit": "deny",
    "write": "deny",
    "bash": "deny",
```

```

    "read": {
      "allow": ["contratos/", "normativas/"],
      "deny": ["confidencial/", "litigios/"]
    }
  }
}

```

⚠ Documentos legales

En entornos legales, considera usar Git para versionar todos los cambios. Así tienes un registro completo de quién accedió a cada documento y cuándo.

1.4. Uso personal

Si usas OpenCode para gestionar documentos personales (fotos familiares, documentos financieros, proyectos privados), puedes proteger tus archivos más sensibles:

```

{
  "permission": {
    "edit": "allow",
    "write": "allow",
    "bash": "deny",
    "read": {
      "allow": ["documentos/", "proyectos/"],
      "deny": ["financiero/", "privado/", "fotos/"]
    }
  }
}

```

Analogía: es como tener diferentes llaves para diferentes habitaciones de tu casa. La llave principal abre todo, pero puedes dar copias parciales a quien necesite acceso limitado.

2. Matriz de permisos por rol

La siguiente tabla muestra qué permisos necesita cada rol en un equipo típico:

Rol	edit	write	bash	read	browser
Estudiante	deny	allow	deny	allow	allow
Profesor	allow	allow	deny	allow	allow
Escritor	allow	allow	deny	allow	allow
Editor	allow	allow	deny	allow	allow
Revisor	deny	deny	deny	allow	deny
Administrador	allow	allow	allow	allow	allow

Cómo leer esta tabla:

- **edit** — ¿puede modificar archivos existentes?
- **write** — ¿puede crear archivos nuevos?
- **bash** — ¿puede ejecutar comandos del sistema?
- **read** — ¿puede leer archivos?
- **browser** — ¿puede navegar por internet?

i Principio de mínimo privilegio

Otorga solo los permisos necesarios para cada tarea. Si un revisor solo necesita leer, no le des permiso de escritura. Si un escritor no necesita ejecutar comandos, deniega bash.

3. Depuración de permisos

Cuando algo no funciona como esperas, sigue estos pasos para diagnosticar el problema:

3.1. Síntoma: “No tengo permiso para editar”

Causa probable: Hay un `edit: deny` en la configuración del proyecto o del agente.

Solución:

1. Revisa `opencode.jsonc` en la raíz del proyecto
2. Revisa el archivo del agente en `.opencode/agents/`
3. Busca cualquier `edit: deny`
4. Modifica o elimina la restricción

3.2. Síntoma: OpenCode no puede leer ciertos archivos

Causa probable: El directorio está en la lista `deny` de `read`.

Solución:

1. Revisa la sección `read` en `opencode.jsonc`
2. Añade el directorio a la lista `allow`
3. Recarga OpenCode

3.3. Síntoma: Bash no funciona aunque está permitido

Causa probable: La configuración global tiene `bash: deny` y prevalece sobre la del proyecto.

Solución:

1. Revisa `~/.config/opencode/opencode.jsonc`
2. Verifica que no tenga `bash: deny`
3. Recuerda: los permisos más restrictivos ganan

3.4. Tabla de diagnóstico rápido

Síntoma	Causa	Solución
No edita archivos	<code>edit: deny</code>	Revisar <code>opencode.jsonc</code> del proyecto
No lee directorios	<code>read.deny</code> contiene el dir	Añadir a <code>read.allow</code>
No ejecuta bash	<code>bash: deny</code>	Verificar config global y proyecto
Agente no funciona	Permiso del agente restrictivo	Revisar <code>.opencode/agents/*.md</code>
Cambios no se guardan	<code>write: deny</code>	Permitir <code>write</code> en config

4. Permisos y Git

Los permisos de OpenCode y Git se complementan: OpenCode controla **quién puede hacer qué** en tiempo real, Git controla **qué se guarda y quién cambió qué**.

4.1. Flujo de trabajo recomendado

1. **Configura permisos** — define qué puede hacer OpenCode
2. **Trabaja normalmente** — OpenCode respeta los permisos
3. **Commitea cambios** — Git guarda una instantánea
4. **Revisa el historial** — Git te muestra quién hizo qué

4.2. Ejemplo práctico

Imagina un equipo que trabaja en un manual:

- El **escritor** crea contenido (permiso: edit + write)
- El **editor** revisa y corrige (permiso: edit + write)
- El **revisor** solo comenta (permiso: read)
- Git guarda cada cambio con el autor y la fecha

Si el revisor intenta modificar un archivo, OpenCode lo deniega. Si alguien borra algo por error, Git lo recupera.

Consejo avanzado

Usa `git diff` después de cada sesión para ver qué cambios hizo OpenCode. Si algo no te gusta, puedes deshacerlo con `git checkout` o `git restore`.

5. Lista de verificación

Antes de configurar permisos, responde estas preguntas:

- ☐ ¿**Quiénes usarán OpenCode?** — estudiantes, profesores, editores, etc.
- ☐ ¿**Qué archivos necesitan protegerse?** — documentos sensibles, datos privados, configuraciones
- ☐ ¿**Quién necesita ejecutar bash?** — solo administradores o también usuarios avanzados
- ☐ ¿**Los permisos son consistentes con Git?** — si usas Git, asegúrate de que los permisos no interfieran
- ☐ ¿**Has probado cada restricción?** — verifica que el comportamiento es el esperado
- ☐ ¿**Documentaste los permisos?** — incluye comentarios explicando por qué existen restricciones

6. Errores comunes

Error	Causa	Solución
“No tengo permiso para editar”	<code>edit: deny</code> en proyecto	Revisar <code>opencode.jsonc</code>
Agente no puede leer archivos	Permiso de agente más restrictivo	Verificar <code>.opencode/agents/*.md</code>
Bash no funciona	<code>deny</code> global prevalece	Verificar config global
OpenCode lee archivos privados	Falta <code>read.deny</code>	Añadir directorios a <code>deny</code>
Cambios no se guardan	<code>write: deny</code>	Permitir <code>write</code> en config
Permisos no se aplican	OpenCode no se recargó	Cerrar y volver a abrir

Recuerda

Los permisos más restrictivos ganan. Si el proyecto deniega bash y el agente lo permite, bash seguirá denegado. Siempre verifica la jerarquía: global → proyecto → agente.

7. Referencias

- [Documentación oficial de permisos de OpenCode](#)
- [Documentación de configuración de OpenCode](#)
- [Principio de mínimo privilegio](#) — concepto de seguridad

Curso completo disponible en: palazon.github.io/ocCursoAvanzado

Curso OpenCode 102 · Idea original de JA Palazón · Junio 2026